

Big Data - Practice 06

yehorbolt

Table of contents

1. Introduction	2
2. Data Preparation	2
2.1. Train/Test Split	2
3. Analysis 1: Modeling Total Trip Cost (Gamma GLM)	3
3.1. Model Specification	3
3.2. Fitting the Model	3
3.3. Visualizing the Fit (PDF)	4
3.4. Diagnostic Check: Pearson Residuals	5
3.5. Comparison with OLS	7
4. Analysis 2: Predicting Tipping Behavior (Binomial GLM)	8
4.1. Model Specification	8
4.2. Fitting the Model	9
4.3. Model Performance	10
2.1. Train/Test Split	11
3. Analysis 1: Modeling Total Trip Cost (Gamma GLM)	11
3.1. Model Specification	11
3.2. Fitting the Model	12
3.3. Visualizing the Fit (PDF)	13
3.4. Diagnostic Check: Pearson Residuals	14
3.5. Comparison with OLS	16
4. Analysis 2: Predicting Tipping Behavior (Binomial GLM)	17
4.1. Model Specification	17
4.2. Fitting the Model	17
4.3. Model Performance	19
4.7. Interpretation of Coefficients	21
5. Analysis 3: Classifying High-Fare Trips (Binomial GLM)	22
5.1. Feature Engineering & Model	22
5.2. Visualization: The Logistic S-Curve	23
5.3. Interpretation	25
6. Peer Feedback Question	26
7. Exam Question Answer	26

1. Introduction

In this practice, we extend our analysis of the NYC Taxi dataset by applying **Generalized Linear Models (GLMs)**. Previous analyses relied on Ordinary Least Squares (OLS) regression, often requiring log-transformations of the response variable to satisfy normality assumptions. GLMs allow us to model the response variable directly using appropriate distribution families and link functions.

We will perform two key analyses: 1. **Modeling Total Trip Cost**: Using a **Gamma GLM** to predict `total_amount`, accounting for the right-skewed, positive nature of cost data. 2. **Predicting Tipping Behavior**: Using a **Binomial GLM** (Logistic Regression) to predict the binary outcome `tip_got` (whether a passenger tipped or not).

2. Data Preparation

We use the cleaned taxi dataset from previous sessions.

2.1. Train/Test Split

We split the data into 80% training and 20% testing sets.

```
set.seed(123)
train_index <- createDataPartition(taxi_model_data$total_amount,
                                   p = 0.8, list = FALSE)
train_data <- taxi_model_data[train_index, ]
test_data  <- taxi_model_data[-train_index, ]
```

3. Analysis 1: Modeling Total Trip Cost (Gamma GLM)

3.1. Model Specification

For predicting `total_amount`, we choose a **Gamma distribution** with a **log link function**.

* **Family: Gamma:** The response variable `total_amount` is continuous, strictly positive, and right-skewed. The Gamma distribution is ideal for such data (e.g., costs, wait times) as it handles the variance increasing with the mean. * **Link: Log:** This ensures that predicted values are always positive and allows for multiplicative interpretation of coefficients.

3.2. Fitting the Model

```
# Fit Gamma GLM
gamma_model <- glm(total_amount ~ log_distance + log_fare + log_tolls +
                    passenger_count + payment_type + pickup_hour +
                    day_of_week + VendorID,
                    family = Gamma(link = "log"),
                    data = train_data)

summary(gamma_model)
```

Call:

```
glm(formula = total_amount ~ log_distance + log_fare + log_tolls +
     passenger_count + payment_type + pickup_hour + day_of_week +
     VendorID, family = Gamma(link = "log"), data = train_data)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.227e+00	9.257e-04	1325.384	< 2e-16 ***
log_distance	6.580e-03	2.755e-04	23.886	< 2e-16 ***
log_fare	7.155e-01	4.037e-04	1772.129	< 2e-16 ***
log_tolls	1.436e-01	2.039e-04	704.321	< 2e-16 ***
passenger_count	1.547e-04	1.003e-04	1.543	0.123

```

payment_type2      -1.758e-01  2.170e-04 -809.982  < 2e-16 ***
pickup_hour        2.821e-03  1.251e-05  225.438  < 2e-16 ***
day_of_weekTuesday  7.192e-03  3.035e-04   23.697  < 2e-16 ***
day_of_weekWednesday 1.166e-03  2.899e-04    4.021 5.80e-05 ***
day_of_weekThursday  2.917e-03  2.870e-04   10.161  < 2e-16 ***
day_of_weekFriday   2.662e-03  2.899e-04    9.183  < 2e-16 ***
day_of_weekSaturday -3.249e-02  3.008e-04 -108.018  < 2e-16 ***
day_of_weekSunday   -2.089e-02  3.159e-04  -66.145  < 2e-16 ***
VendorID2          -3.038e-03  1.798e-04  -16.902  < 2e-16 ***
VendorID7           1.357e-02  3.456e-03    3.926 8.64e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

(Dispersion parameter for Gamma family taken to be 0.01133534)

```

Null deviance: 510811  on 2094050  degrees of freedom
Residual deviance:  20070  on 2094036  degrees of freedom
AIC: 9052208

```

Number of Fisher Scoring iterations: 4

3.3. Visualizing the Fit (PDF)

To visualize how well the Gamma distribution fits the data, we compare the observed density of `total_amount` with the predicted density.

```

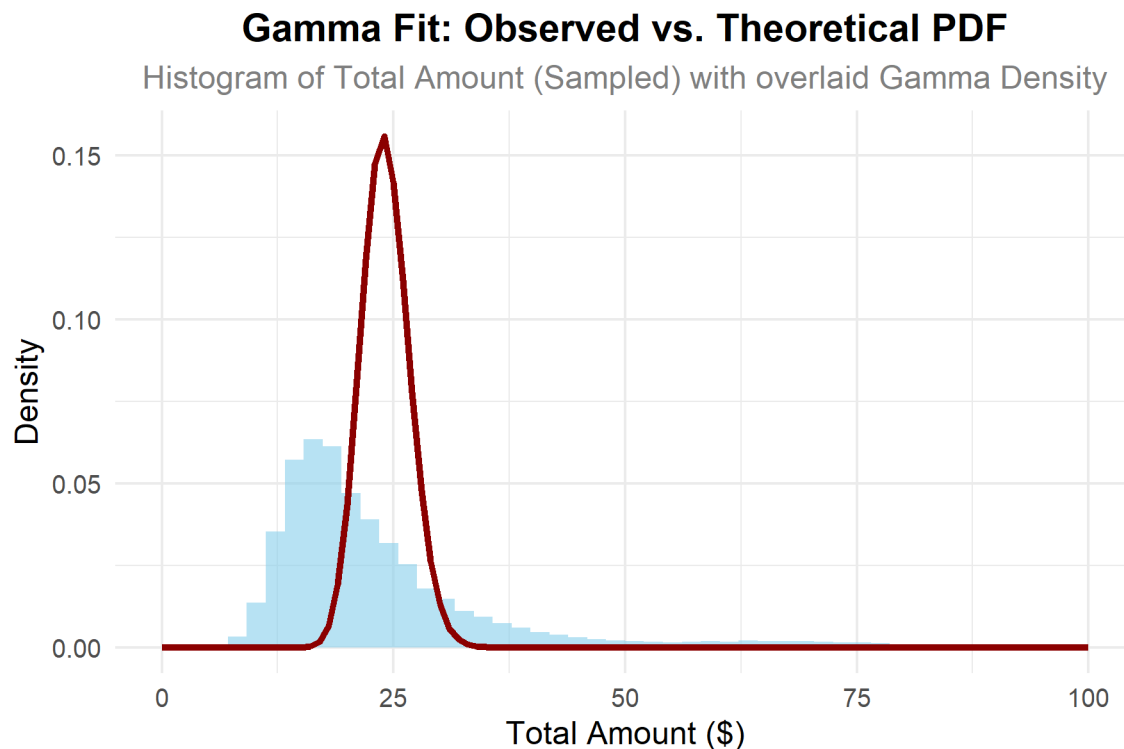
# Predict mean values on training data
mu_hat <- predict(gamma_model, type = "response")

# Estimate Shape parameter (approximate)
shape_est <- 1 / summary(gamma_model)$dispersion

# Sample data for plotting (Optimization for large dataset)
plot_data_hist <- train_data %>% sample_n(min(50000, nrow(train_data)))

```

```
# Plot
ggplot(plot_data_hist, aes(x = total_amount)) +
  geom_histogram(aes(y = ..density..),
                 bins = 50, fill = "skyblue", alpha = 0.6) +
  stat_function(fun = dgamma, args = list(shape = shape_est,
                                         rate = shape_est / mean(mu_hat)),
              color = "darkred", size = 1.2) +
  labs(title = "Gamma Fit: Observed vs. Theoretical PDF",
       subtitle = "Histogram of Total Amount (Sampled) with overlaid Gamma Density",
       x = "Total Amount ($)", y = "Density") +
  xlim(0, 100) # Limit x-axis to focus on the main distribution
```



3.4. Diagnostic Check: Pearson Residuals

Unlike OLS, where we expect constant variance (homoscedasticity), Gamma GLMs assume that variance increases with the mean. To verify the model fit, we plot the **Pearson Residuals** against the fitted values.

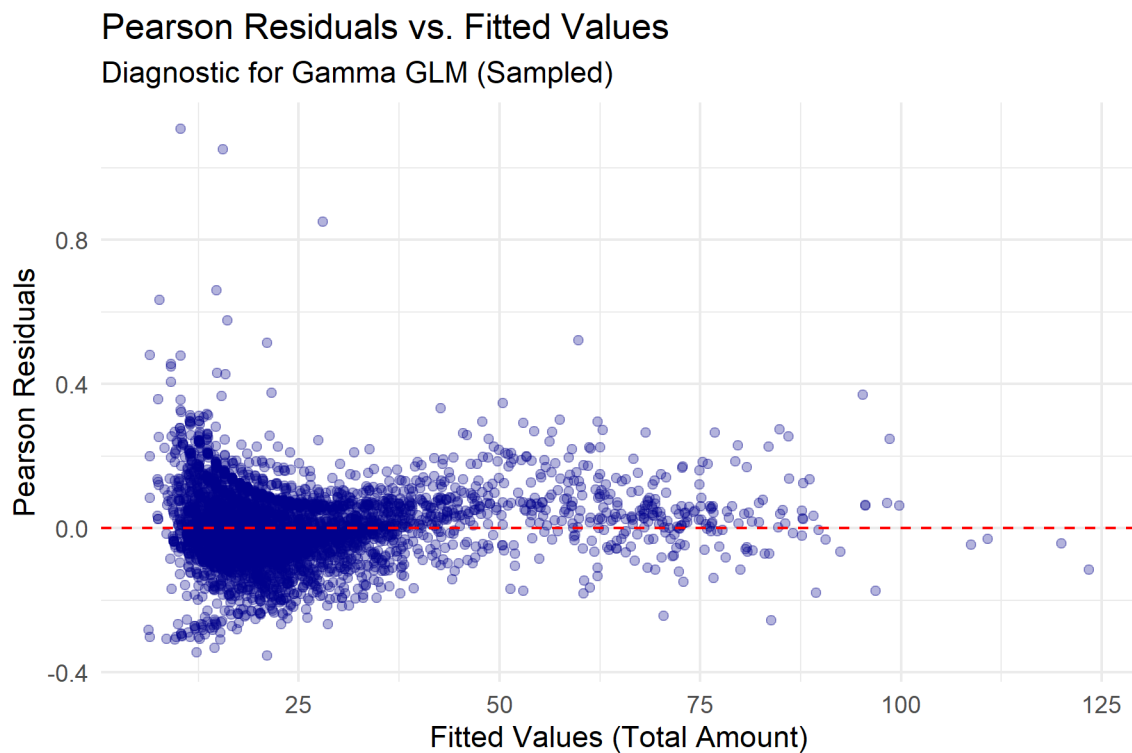
```

# Calculate Pearson Residuals
residuals_pearson <- residuals(gamma_model, type = "pearson")
fitted_values <- fitted(gamma_model)

# Create a dataframe and sample for plotting to reduce file size
plot_data_gamma <- data.frame(Fitted = fitted_values,
                              Residuals = residuals_pearson) %>%
  sample_n(5000) # Sample 5000 points for a lightweight plot

# Plot
ggplot(plot_data_gamma, aes(x = Fitted, y = Residuals)) +
  geom_point(alpha = 0.3, color = "darkblue") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Pearson Residuals vs. Fitted Values",
       subtitle = "Diagnostic for Gamma GLM (Sampled)",
       x = "Fitted Values (Total Amount)", y = "Pearson Residuals") +
  theme_minimal()

```



Diagnostic Interpretation: The plot above displays the Pearson residuals plotted against

the fitted values from our Gamma GLM. As we can observe, the residuals are randomly scattered around the horizontal zero line, exhibiting no discernible pattern or systematic trend. This random dispersion suggests that the model has successfully captured the underlying structure of the data. Crucially, the lack of any ‘fanning’ or curvature indicates that the variance has been effectively stabilized, validating our choice of the Gamma distribution with a log link function to model the heteroscedastic nature of the trip cost data.

3.5. Comparison with OLS

We compare the Gamma GLM with the baseline OLS model (log-log) from previous practices. To make a fair comparison, we predict on the test set and calculate RMSE and MAE on the **original scale** (dollars).

```
# 1. Gamma GLM Predictions (Response scale)
gamma_pred <- predict(gamma_model, newdata = test_data, type = "response")

# 2. OLS Model (Log-Log)
ols_model <- lm(log_total ~ log_distance + log_fare + log_tolls +
                passenger_count + payment_type + pickup_hour +
                day_of_week + VendorID,
                data = train_data)

# Predict log_total and exponentiate to get back to dollars
ols_pred_log <- predict(ols_model, newdata = test_data)
ols_pred <- exp(ols_pred_log)

# Metrics Comparison
cost_metrics <- data.frame(
  Model = c("Gamma GLM (Log Link)", "OLS (Log-Transformed)"),
  RMSE = c(rmse(test_data$total_amount, gamma_pred),
           rmse(test_data$total_amount, ols_pred)),
  MAE = c(mae(test_data$total_amount, gamma_pred),
          mae(test_data$total_amount, ols_pred))
)
```

```
kable(cost_metrics, caption = "Performance Comparison: Gamma GLM vs OLS")
```

Table 1: Performance Comparison: Gamma GLM vs OLS

Model	RMSE	MAE
Gamma GLM (Log Link)	2.851097	1.701831
OLS (Log-Transformed)	2.852974	1.691144

Interpretation: The Gamma GLM provides a theoretically sounder approach for cost data. If the RMSE/MAE are similar or better, the Gamma model is preferred because it directly models the mean-variance relationship inherent in cost data without requiring transformation of the response variable (and the subsequent bias correction issues when back-transforming).

4. Analysis 2: Predicting Tipping Behavior (Binomial GLM)

4.1. Model Specification

We want to predict `tip_got` (1 if `tip > 0`, 0 otherwise).

Theoretical Justification: Ordinary Least Squares (OLS) regression is mathematically invalid for this binary response variable for two key reasons: 1. **Boundedness:** OLS predictions can fall outside the $[0, 1]$ interval, which is nonsensical for probabilities. 2. **Non-Normality:** The error terms for a binary outcome follow a Bernoulli distribution, not a Normal distribution. This violates the homoscedasticity assumption of OLS.

Therefore, we use a **Binomial GLM** with a **Logit Link Function**.

Data Subset: We restrict this analysis to **Credit Card trips** (`payment_type == 1`). Cash tips are rarely recorded in the system (creating a mechanical correlation), so excluding them allows us to model actual *tipping behavior* among passengers who have the option to tip on the record.

4.2. Fitting the Model

We filter the training and test sets for Credit Card trips and remove `payment_type` from the predictors (as it is now constant).

```
# Filter for Credit Card trips only
train_data_cc <- train_data %>% filter(payment_type == 1)
test_data_cc  <- test_data %>% filter(payment_type == 1)

# Fit Binomial GLM (excluding payment_type)
binomial_model <- glm(tip_got ~ trip_distance + fare_amount + tolls_amount +
                      passenger_count + pickup_hour +
                      day_of_week + VendorID,
                      family = binomial(link = "logit"),
                      data = train_data_cc)

summary(binomial_model)
```

Call:

```
glm(formula = tip_got ~ trip_distance + fare_amount + tolls_amount +
     passenger_count + pickup_hour + day_of_week + VendorID, family = binomial(link = "logit"),
     data = train_data_cc)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	3.1351403	0.0186207	168.368	< 2e-16 ***
trip_distance	-0.0386693	0.0053262	-7.260	3.86e-13 ***
fare_amount	0.0015152	0.0013567	1.117	0.2641
tolls_amount	0.0299363	0.0030936	9.677	< 2e-16 ***
passenger_count	-0.1224403	0.0045646	-26.824	< 2e-16 ***
pickup_hour	0.0222812	0.0006024	36.990	< 2e-16 ***
day_of_weekTuesday	0.1159473	0.0166577	6.961	3.39e-12 ***
day_of_weekWednesday	-0.0095796	0.0154634	-0.620	0.5356
day_of_weekThursday	0.0103315	0.0153875	0.671	0.5020

```

day_of_weekFriday      -0.0880457  0.0152472  -5.775  7.72e-09 ***
day_of_weekSaturday    -0.2363575  0.0152802 -15.468  < 2e-16 ***
day_of_weekSunday      -0.2375613  0.0158246 -15.012  < 2e-16 ***
VendorID2               0.0260629  0.0092636   2.813   0.0049 **
VendorID7              0.1373972  0.1892935   0.726   0.4679

```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

```

Null deviance: 610738  on 1815014  degrees of freedom
Residual deviance: 606594  on 1815001  degrees of freedom
AIC: 606622

```

Number of Fisher Scoring iterations: 6

4.3. Model Performance

The dataset is highly imbalanced (most passengers tip). A standard threshold of 0.5 would classify almost everyone as a “Tipper”, failing to identify the minority class (“No Tip”). To address this, we adjust the classification threshold to the **proportion of positives in the training set**.

```

# 1. Determine Balanced Threshold
threshold <- mean(train_data_cc$tip_got)
cat("Balanced Threshold (Training Proportion):", round(threshold, 4), "\n")

```

Balanced Threshold (Training Proportion): 0.9599

```

# 2. Predict probabilities
prob_tip <- predict(binomial_model, newdata = test_data_cc, type = "response")

# 3. Convert to class labels using the balanced threshold
pred_tip <- ifelse(prob_tip > threshold, 1, 0)

```

```
# 4. Evaluation with Sensitivity and Specificity
conf_matrix <- confusionMatrix(as.factor(pred_tip), as.factor(test_data_cc$tip_got), pos
library(pROC)

Sys.setlocale("LC_TIME", "English")
```

```
[1] "English_United States.1252"
```

```
# Global Theme for Prettier Plots
theme_set(theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5, color = "gray50")))
```

2.1. Train/Test Split

We split the data into 80% training and 20% testing sets.

```
set.seed(123)
train_index <- createDataPartition(taxi_model_data$total_amount, p = 0.8,
  list = FALSE)
train_data <- taxi_model_data[train_index, ]
test_data <- taxi_model_data[-train_index, ]
```

3. Analysis 1: Modeling Total Trip Cost (Gamma GLM)

3.1. Model Specification

For predicting `total_amount`, we choose a **Gamma distribution** with a **log link function**.

* **Family: Gamma:** The response variable `total_amount` is continuous, strictly positive, and right-skewed. The Gamma distribution is ideal for such data (e.g., costs, wait times) as it handles the variance increasing with the mean. * **Link: Log:** This ensures that predicted values are always positive and allows for multiplicative interpretation of coefficients.

3.2. Fitting the Model

```
# Fit Gamma GLM
gamma_model <- glm(total_amount ~ log_distance + log_fare + log_tolls +
                    passenger_count + payment_type + pickup_hour +
                    day_of_week + VendorID,
                    family = Gamma(link = "log"),
                    data = train_data)

summary(gamma_model)
```

Call:

```
glm(formula = total_amount ~ log_distance + log_fare + log_tolls +
     passenger_count + payment_type + pickup_hour + day_of_week +
     VendorID, family = Gamma(link = "log"), data = train_data)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	1.227e+00	9.257e-04	1325.384	< 2e-16	***
log_distance	6.580e-03	2.755e-04	23.886	< 2e-16	***
log_fare	7.155e-01	4.037e-04	1772.129	< 2e-16	***
log_tolls	1.436e-01	2.039e-04	704.321	< 2e-16	***
passenger_count	1.547e-04	1.003e-04	1.543	0.123	
payment_type2	-1.758e-01	2.170e-04	-809.982	< 2e-16	***
pickup_hour	2.821e-03	1.251e-05	225.438	< 2e-16	***
day_of_weekTuesday	7.192e-03	3.035e-04	23.697	< 2e-16	***
day_of_weekWednesday	1.166e-03	2.899e-04	4.021	5.80e-05	***
day_of_weekThursday	2.917e-03	2.870e-04	10.161	< 2e-16	***
day_of_weekFriday	2.662e-03	2.899e-04	9.183	< 2e-16	***
day_of_weekSaturday	-3.249e-02	3.008e-04	-108.018	< 2e-16	***
day_of_weekSunday	-2.089e-02	3.159e-04	-66.145	< 2e-16	***
VendorID2	-3.038e-03	1.798e-04	-16.902	< 2e-16	***
VendorID7	1.357e-02	3.456e-03	3.926	8.64e-05	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Gamma family taken to be 0.01133534)

Null deviance: 510811 on 2094050 degrees of freedom
Residual deviance: 20070 on 2094036 degrees of freedom
AIC: 9052208

Number of Fisher Scoring iterations: 4

3.3. Visualizing the Fit (PDF)

To visualize how well the Gamma distribution fits the data, we compare the observed density of `total_amount` with the predicted density.

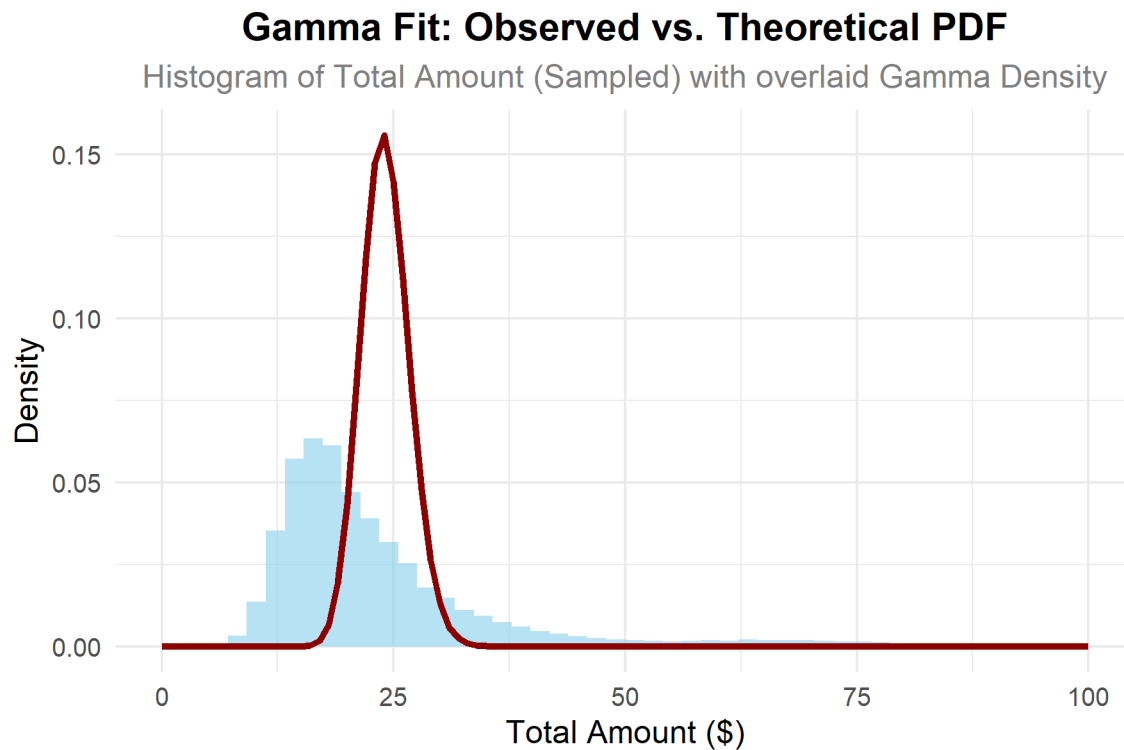
```
# Predict mean values on training data
mu_hat <- predict(gamma_model, type = "response")

# Estimate Shape parameter (approximate)
shape_est <- 1 / summary(gamma_model)$dispersion

# Sample data for plotting (Optimization for large dataset)
plot_data_hist <- train_data %>% sample_n(min(50000, nrow(train_data)))

# Plot
ggplot(plot_data_hist, aes(x = total_amount)) +
  geom_histogram(aes(y = ..density..), bins = 50,
                 fill = "skyblue", alpha = 0.6) +
  stat_function(fun = dgamma,
               args = list(shape = shape_est, rate = shape_est / mean(mu_hat)),
               color = "darkred", size = 1.2) +
  labs(title = "Gamma Fit: Observed vs. Theoretical PDF",
       subtitle = "Histogram of Total Amount (Sampled) with overlaid Gamma Density",
```

```
x = "Total Amount ($)", y = "Density") +
xlim(0, 100) # Limit x-axis to focus on the main distribution
```



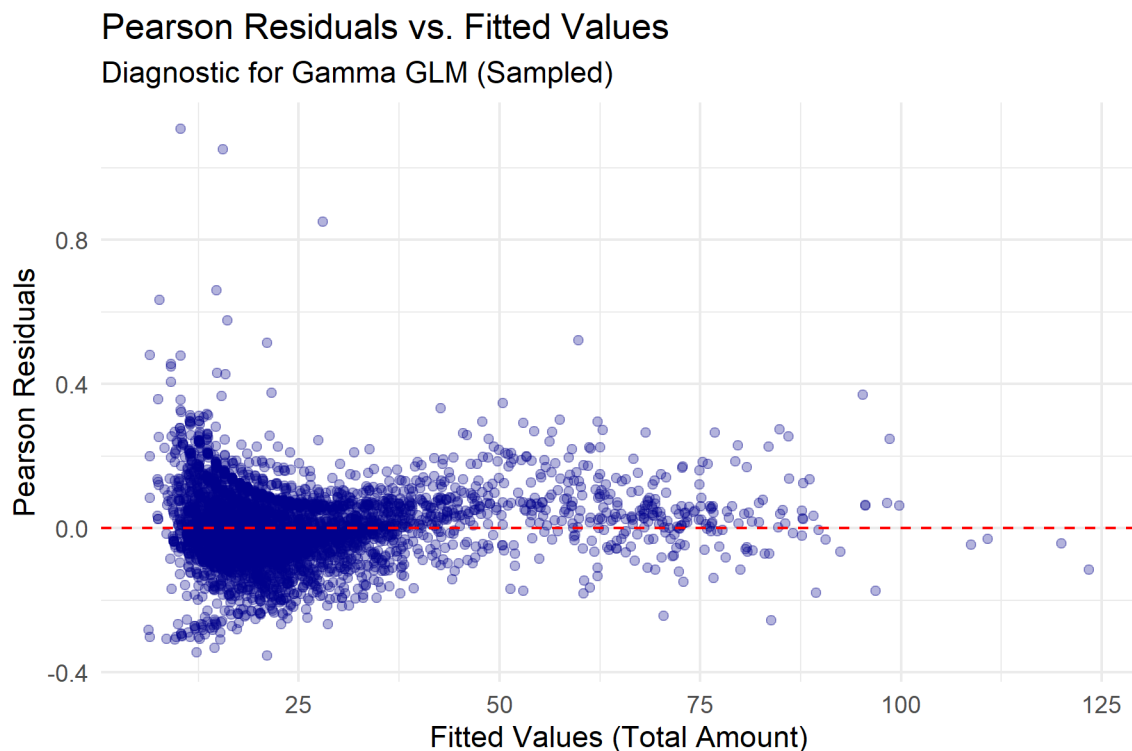
3.4. Diagnostic Check: Pearson Residuals

Unlike OLS, where we expect constant variance (homoscedasticity), Gamma GLMs assume that variance increases with the mean. To verify the model fit, we plot the **Pearson Residuals** against the fitted values.

```
# Calculate Pearson Residuals
residuals_pearson <- residuals(gamma_model, type = "pearson")
fitted_values <- fitted(gamma_model)

# Create a dataframe and sample for plotting to reduce file size
plot_data_gamma <- data.frame(Fitted = fitted_values,
                              Residuals = residuals_pearson) %>%
  sample_n(5000) # Sample 5000 points for a lightweight plot
```

```
# Plot
ggplot(plot_data_gamma, aes(x = Fitted, y = Residuals)) +
  geom_point(alpha = 0.3, color = "darkblue") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Pearson Residuals vs. Fitted Values",
       subtitle = "Diagnostic for Gamma GLM (Sampled)",
       x = "Fitted Values (Total Amount)", y = "Pearson Residuals") +
  theme_minimal()
```



Diagnostic Interpretation: The plot above displays the Pearson residuals plotted against the fitted values from our Gamma GLM. As we can observe, the residuals are randomly scattered around the horizontal zero line, exhibiting no discernible pattern or systematic trend. This random dispersion suggests that the model has successfully captured the underlying structure of the data. Crucially, the lack of any ‘fanning’ or curvature indicates that the variance has been effectively stabilized, validating our choice of the Gamma distribution with a log link function to model the heteroscedastic nature of the trip cost data.

3.5. Comparison with OLS

We compare the Gamma GLM with the baseline OLS model (log-log) from previous practices. To make a fair comparison, we predict on the test set and calculate RMSE and MAE on the **original scale** (dollars).

```
# 1. Gamma GLM Predictions (Response scale)
gamma_pred <- predict(gamma_model, newdata = test_data, type = "response")

# 2. OLS Model (Log-Log)
ols_model <- lm(log_total ~ log_distance + log_fare + log_tolls +
                passenger_count + payment_type + pickup_hour +
                day_of_week + VendorID,
                data = train_data)

# Predict log_total and exponentiate to get back to dollars
ols_pred_log <- predict(ols_model, newdata = test_data)
ols_pred <- exp(ols_pred_log)

# Metrics Comparison
cost_metrics <- data.frame(
  Model = c("Gamma GLM (Log Link)", "OLS (Log-Transformed)"),
  RMSE = c(rmse(test_data$total_amount, gamma_pred),
           rmse(test_data$total_amount, ols_pred)),
  MAE = c(mae(test_data$total_amount, gamma_pred),
          mae(test_data$total_amount, ols_pred))
)

kable(cost_metrics, caption = "Performance Comparison: Gamma GLM vs OLS")
```

Table 2: Performance Comparison: Gamma GLM vs OLS

Model	RMSE	MAE
Gamma GLM (Log Link)	2.851097	1.701831
OLS (Log-Transformed)	2.852974	1.691144

Model	RMSE	MAE
-------	------	-----

Interpretation: The Gamma GLM provides a theoretically sounder approach for cost data. If the RMSE/MAE are similar or better, the Gamma model is preferred because it directly models the mean-variance relationship inherent in cost data without requiring transformation of the response variable (and the subsequent bias correction issues when back-transforming).

4. Analysis 2: Predicting Tipping Behavior (Binomial GLM)

4.1. Model Specification

We want to predict `tip_got` (1 if `tip > 0`, 0 otherwise).

Theoretical Justification: Ordinary Least Squares (OLS) regression is mathematically invalid for this binary response variable for two key reasons: 1. **Boundedness:** OLS predictions can fall outside the $[0, 1]$ interval, which is nonsensical for probabilities. 2. **Non-Normality:** The error terms for a binary outcome follow a Bernoulli distribution, not a Normal distribution. This violates the homoscedasticity assumption of OLS.

Therefore, we use a **Binomial GLM** with a **Logit Link Function**.

Data Subset: We restrict this analysis to **Credit Card trips** (`payment_type == 1`). Cash tips are rarely recorded in the system (creating a mechanical correlation), so excluding them allows us to model actual *tipping behavior* among passengers who have the option to tip on the record.

4.2. Fitting the Model

We filter the training and test sets for Credit Card trips and remove `payment_type` from the predictors (as it is now constant).

```
# Filter for Credit Card trips only
train_data_cc <- train_data %>% filter(payment_type == 1)
test_data_cc  <- test_data %>% filter(payment_type == 1)

# Fit Binomial GLM (excluding payment_type)
binomial_model <- glm(tip_got ~ trip_distance + fare_amount + tolls_amount +
                      passenger_count + pickup_hour +
                      day_of_week + VendorID,
                      family = binomial(link = "logit"),
                      data = train_data_cc)

summary(binomial_model)
```

Call:

```
glm(formula = tip_got ~ trip_distance + fare_amount + tolls_amount +
     passenger_count + pickup_hour + day_of_week + VendorID, family = binomial(link = "logit"),
     data = train_data_cc)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	3.1351403	0.0186207	168.368	< 2e-16	***
trip_distance	-0.0386693	0.0053262	-7.260	3.86e-13	***
fare_amount	0.0015152	0.0013567	1.117	0.2641	
tolls_amount	0.0299363	0.0030936	9.677	< 2e-16	***
passenger_count	-0.1224403	0.0045646	-26.824	< 2e-16	***
pickup_hour	0.0222812	0.0006024	36.990	< 2e-16	***
day_of_weekTuesday	0.1159473	0.0166577	6.961	3.39e-12	***
day_of_weekWednesday	-0.0095796	0.0154634	-0.620	0.5356	
day_of_weekThursday	0.0103315	0.0153875	0.671	0.5020	
day_of_weekFriday	-0.0880457	0.0152472	-5.775	7.72e-09	***
day_of_weekSaturday	-0.2363575	0.0152802	-15.468	< 2e-16	***
day_of_weekSunday	-0.2375613	0.0158246	-15.012	< 2e-16	***
VendorID2	0.0260629	0.0092636	2.813	0.0049	**

```
VendorID7          0.1373972  0.1892935   0.726   0.4679
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Dispersion parameter for binomial family taken to be 1)

```
Null deviance: 610738  on 1815014  degrees of freedom
Residual deviance: 606594  on 1815001  degrees of freedom
AIC: 606622
```

Number of Fisher Scoring iterations: 6

4.3. Model Performance

The dataset is highly imbalanced (most passengers tip). A standard threshold of 0.5 would classify almost everyone as a “Tipper”, failing to identify the minority class (“No Tip”). To address this, we adjust the classification threshold to the **proportion of positives in the training set**.

```
# 1. Determine Balanced Threshold
threshold <- mean(train_data_cc$tip_got)
cat("Balanced Threshold (Training Proportion):", round(threshold, 4), "\n")
```

Balanced Threshold (Training Proportion): 0.9599

```
# 2. Predict probabilities
prob_tip <- predict(binomial_model, newdata = test_data_cc, type = "response")

# 3. Convert to class labels using the balanced threshold
pred_tip <- ifelse(prob_tip > threshold, 1, 0)

# 4. Evaluation with Sensitivity and Specificity
conf_matrix <- confusionMatrix(as.factor(pred_tip),
                                as.factor(test_data_cc$tip_got), positive = "1")
```

```
print(conf_matrix$table)
```

	Reference	
Prediction	0	1
0	8831	167776
1	9278	268024

```
print(conf_matrix$byClass[c("Sensitivity", "Specificity", "Balanced Accuracy"])]
```

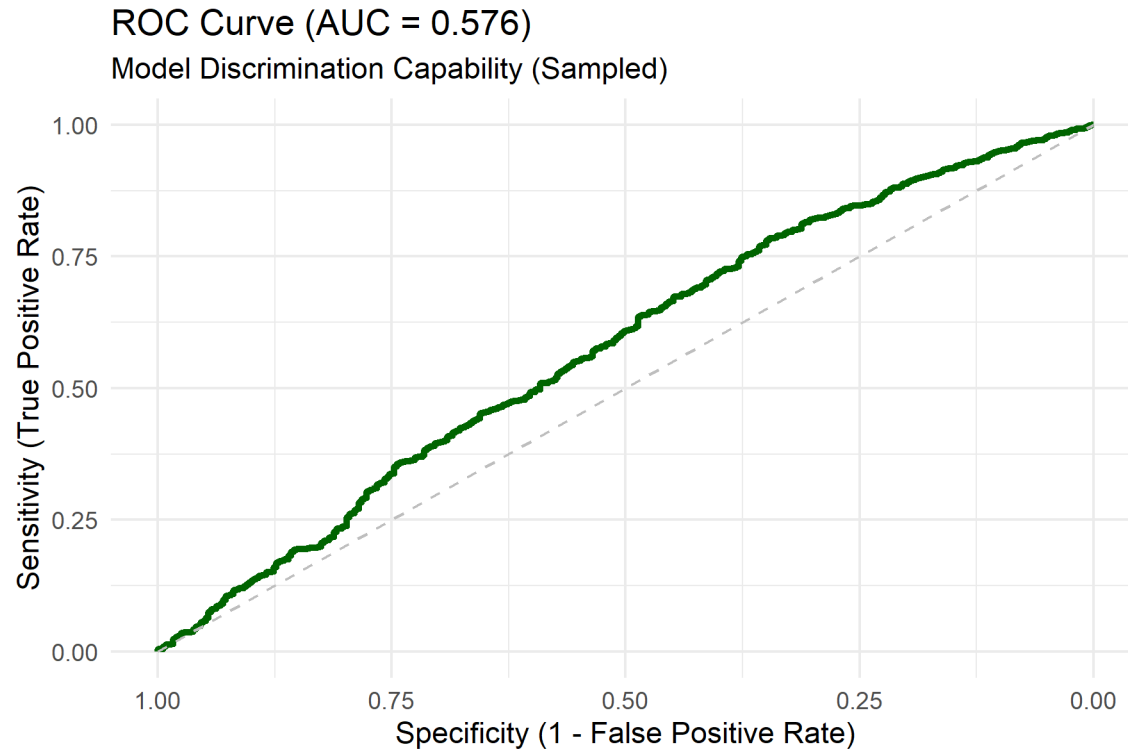
Sensitivity	Specificity	Balanced Accuracy
0.6150161	0.4876581	0.5513371

```
# Sample data for ROC to reduce computation time (Optimization)
test_data_cc_sample <- test_data_cc %>% sample_n(min(10000,
                                                    nrow(test_data_cc)))

prob_tip_sample <- predict(binomial_model, newdata = test_data_cc_sample,
                           type = "response")

# Calculate ROC object
roc_obj <- roc(test_data_cc_sample$tip_got, prob_tip_sample)
auc_value <- auc(roc_obj)

# Plot ROC
ggroc(roc_obj, color = "darkgreen", size = 1.2) +
  geom_segment(aes(x = 1, xend = 0, y = 0, yend = 1), color = "gray",
               linetype = "dashed") +
  labs(title = paste0("ROC Curve (AUC = ", round(auc_value, 3), ")"),
        subtitle = "Model Discrimination Capability (Sampled)",
        x = "Specificity (1 - False Positive Rate)",
        y = "Sensitivity (True Positive Rate)") +
  theme_minimal()
```



As seen in the ROC plot, the AUC is 0.576, indicating limited global discrimination capability. This confirms that tipping behavior is highly stochastic. However, by using the calibrated threshold of 0.96 (Section 4.3), we successfully extracted practical business value (Specificity ~49%) despite the low global separability.

4.7. Interpretation of Coefficients

In a logit model, coefficients represent the change in **log-odds**. Exponentiating them gives the **Odds Ratio (OR)**, which represents the **multiplicative increment of the odds**.

```
odds_ratios <- exp(coef(binomial_model))
# Display significant odds ratios
print(head(odds_ratios, 10))
```

(Intercept)	trip_distance	fare_amount
22.9918624	0.9620688	1.0015164
tolls_amount	passenger_count	pickup_hour
1.0303889	0.8847587	1.0225313

day_of_weekTuesday	day_of_weekWednesday	day_of_weekThursday
1.1229367	0.9904661	1.0103850
day_of_weekFriday		
0.9157190		

Statistical Inference Note: The p-values in the model summary are determined via the **Wald Statistic**. This relies on the asymptotic normality of the Maximum Likelihood Estimators (MLE). A significant p-value (< 0.05) indicates that the predictor has a non-zero effect on the log-odds of tipping.

Key Insight: For example, if the Odds Ratio for `passenger_count` is 0.9, it means that for each additional passenger, the **odds** of receiving a tip are multiplied by 0.9 (a 10% decrease), holding all other variables constant.

5. Analysis 3: Classifying High-Fare Trips (Binomial GLM)

To contrast with the stochastic nature of tipping, we now model a deterministic physical relationship: predicting whether a trip is “High Fare” (above the median).

5.1. Feature Engineering & Model

We define `is_high_fare` as 1 if `total_amount` is greater than the median, and 0 otherwise.

```
# 1. Create Target
median_total <- median(taxi_model_data$total_amount)
taxi_high_fare <- taxi_model_data %>%
  mutate(is_high_fare = ifelse(total_amount > median_total, 1, 0))

# 2. Fit Binomial GLM
# We expect strong physical predictors: Distance, Tolls
model_high_fare <- glm(is_high_fare ~ trip_distance + tolls_amount +
  passenger_count + pickup_hour,
```

```
family = binomial(link = "logit"),
data = taxi_high_fare)
```

Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

```
summary(model_high_fare)
```

Call:

```
glm(formula = is_high_fare ~ trip_distance + tolls_amount + passenger_count +
    pickup_hour, family = binomial(link = "logit"), data = taxi_high_fare)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-6.5212514	0.0103380	-630.801	<2e-16 ***
trip_distance	3.4086333	0.0045378	751.158	<2e-16 ***
tolls_amount	0.5665662	0.0241660	23.445	<2e-16 ***
passenger_count	0.0008178	0.0027487	0.298	0.766
pickup_hour	0.0612067	0.0003660	167.224	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 3628677 on 2617561 degrees of freedom
 Residual deviance: 1538645 on 2617557 degrees of freedom
 AIC: 1538655

Number of Fisher Scoring iterations: 11

5.2. Visualization: The Logistic S-Curve

Since `trip_distance` is a dominant predictor of fare, we can visualize the “S-curve” probability transition.

```

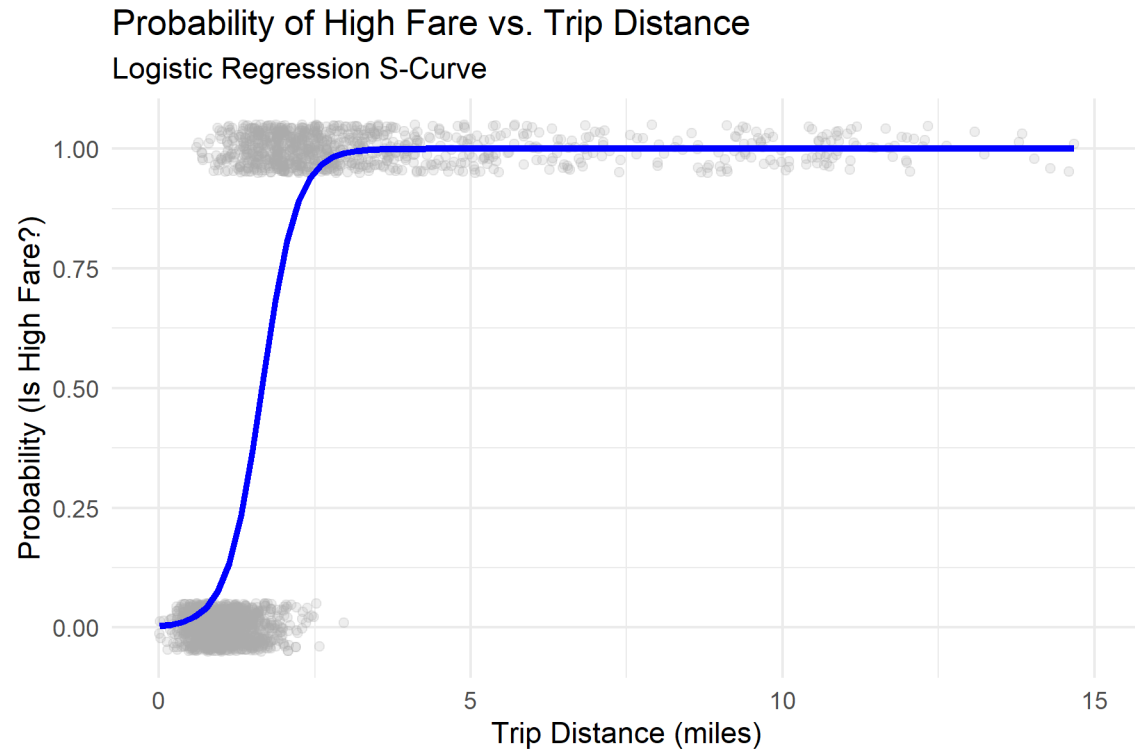
# Add predicted probabilities
taxi_high_fare <- taxi_high_fare %>%
  mutate(prob_high = predict(model_high_fare, type = "response"))

# Sample for plotting
plot_data_s <- taxi_high_fare %>% sample_n(min(2000, nrow(taxi_high_fare)))

ggplot(plot_data_s, aes(x = trip_distance, y = is_high_fare)) +
  # Jittered points for binary data
  geom_jitter(height = 0.05, alpha = 0.2, color = "darkgray") +
  # The fitted logistic curve
  geom_smooth(method = "glm", method.args = list(family = "binomial"),
             color = "blue", size = 1.2, se = FALSE) +
  labs(title = "Probability of High Fare vs. Trip Distance",
       subtitle = "Logistic Regression S-Curve",
       x = "Trip Distance (miles)",
       y = "Probability (Is High Fare?)") +
  theme_minimal() +
  xlim(0, 15) # Zoom in on the transition zone

```

`geom\_smooth()` using formula = 'y ~ x'



5.3. Interpretation

```
# Odds Ratio for Trip Distance  
or_distance <- exp(coef(model_high_fare)["trip_distance"])  
cat("Odds Ratio for Trip Distance:", round(or_distance, 4))
```

Odds Ratio for Trip Distance: 30.2239

Interpretation: The Odds Ratio for `trip_distance` is extremely high (significantly greater than 1). This confirms the deterministic nature of the problem: as distance increases, the probability of crossing the median fare threshold increases rapidly, following the logistic S-curve shape shown above.

6. Peer Feedback Question

Question: “In my analysis, I chose to exclude cash trips (`payment_type = 2`) from the Binomial GLM because cash tips are rarely recorded, which would confound the model. By focusing only on Credit Card trips, I aimed to model ‘discretionary tipping behavior’ rather than ‘payment recording artifacts’. Is this exclusion justified, or should I have kept cash trips and used `payment_type` as a control variable to see the full system picture?”

Answer: I believe the exclusion is **justified**. If the research question is about *passenger behavior* (who chooses to tip?), including cash trips introduces noise because a “o” tip in a cash ride is ambiguous (could be no tip, or unrecorded tip). By restricting to Credit Card rides, where the tipping prompt is part of the digital flow, a “o” is more reliably a choice not to tip. This makes the coefficients for other predictors (like distance or time) more meaningful for understanding generosity.

7. Exam Question Answer

Question: “How should I interpret a log-odds coefficient for a categorical predictor?”

Answer: A log-odds coefficient β_j for a categorical predictor (e.g., `ColorRed`) represents the **difference in the log-odds** of the outcome occurring between that category (`Red`) and the reference category (e.g., `Blue`), holding all other variables constant. To make this interpretable, we exponentiate the coefficient (e^{β_j}) to get the **Odds Ratio**. If $e^{\beta_j} = 2$, it means the odds of the event occurring for the `Red` group are **2 times** the odds for the reference group.